

Arquitectura Integral y Análisis de Viabilidad para Sistemas de UI Dirigidos por Servidor (SDUI) con Lógica de Negocio Dinámica en Android Nativo (Jetpack Compose)

1. Resumen Ejecutivo y Alcance del Análisis

El presente informe técnico tiene como objetivo evaluar exhaustivamente la viabilidad, las implicaciones arquitectónicas y los desafíos de implementación para el desarrollo de una aplicación Android nativa utilizando Kotlin y Jetpack Compose, bajo un paradigma estricto de **Interfaz de Usuario Dirigida por Servidor (SDUI)**. El requerimiento central estipula que la aplicación debe construirse en tiempo de ejecución consumiendo servicios que proveen no solo los datos, sino la configuración completa de los módulos de interfaz, la interacción y, crucialmente, la **lógica de negocio** mediante un esquema JSON.

El análisis confirma que la arquitectura propuesta es viable y representa la vanguardia del desarrollo móvil moderno, adoptada por corporaciones tecnológicas de alto perfil como Airbnb, Lyft y Block (anteriormente Square) para resolver el problema de la latencia en la distribución de actualizaciones.¹ La transición de un modelo "Client-Driven" a uno "Server-Driven" permite iteraciones inmediatas sin pasar por los procesos de revisión de Google Play Store. Sin embargo, la inyección de lógica de negocio compleja presenta desafíos técnicos significativos que requieren la integración de motores de ejecución de código dinámico (como Zipline) o motores de reglas declarativas (como JsonLogic), operando dentro de los límites estrictos de seguridad y políticas de las tiendas de aplicaciones.⁴

Este documento detalla una arquitectura de referencia que utiliza un "Motor de Renderizado" (*Player*) en el cliente, capaz de interpretar esquemas JSON agnósticos para componer vistas nativas, gestionar estados reactivos y ejecutar lógica segura, proporcionando una hoja de ruta técnica para su implementación.

2. El Cambio de Paradigma: De la Compilación Estática a la Composición Dinámica

2.1. Limitaciones del Modelo Tradicional Client-Driven

En el desarrollo convencional de Android, la arquitectura se basa en un acoplamiento fuerte entre la lógica de presentación y el binario compilado. Los archivos de diseño (XML o funciones Composable), la navegación y las reglas de negocio se empaquetan dentro del APK o AAB. El servidor actúa meramente como un proveedor de datos de dominio (por ejemplo, un objeto JSON representando un Producto con precio y nombre), y es responsabilidad exclusiva del cliente saber cómo transformar esos datos en píxeles.⁶

Este modelo presenta fricciones operativas críticas:

1. **Latencia de Despliegue:** Cualquier cambio en la UI, por trivial que sea (como cambiar el color de un botón o reordenar una lista), requiere un nuevo ciclo de compilación, pruebas, envío a la tienda, revisión (que puede tomar días) y la actualización por parte del usuario final.
2. **Fragmentación de Versiones:** Es imposible garantizar que todos los usuarios vean la misma interfaz o experimenten la misma lógica de negocio simultáneamente, ya que la base de usuarios está dispersa en múltiples versiones de la aplicación.
3. **Duplicación de Lógica:** En entornos multiplataforma (Android, iOS, Web), la lógica de presentación a menudo se reimplementa tres veces, aumentando el riesgo de inconsistencias.⁸

2.2. La Promesa del Server-Driven UI (SDUI)

El SDUI invierte la responsabilidad: el servidor no envía datos de dominio, sino **instrucciones de renderizado**. La aplicación móvil se transforma en un "navegador nativo" o un motor de renderizado "tonto" (*dumb renderer*), cuya única función es interpretar y pintar lo que el servidor le dicta.

En este esquema, el JSON de respuesta no dice "aquí tienes un producto", sino "dibuja una tarjeta con una imagen arriba, un texto en negrita debajo y un botón verde al final". Esto permite cambios instantáneos en la estructura y comportamiento de la aplicación sin actualizaciones de binarios.³

2.3. Jetpack Compose como Catalizador Tecnológico

Históricamente, implementar SDUI en Android era una tarea titánica debido a la naturaleza imperativa del sistema de Vistas (Android View System). Manipular el árbol de vistas, gestionar el reciclaje en RecyclerView y manejar los estados mutables requería bibliotecas complejas como Epoxy o Litho para abstraer la dificultad.⁶

Jetpack Compose, el moderno kit de herramientas de UI declarativo de Google, se alinea conceptualmente de manera perfecta con el SDUI. En Compose, la UI se describe como funciones inmutables de datos: $UI = f(\text{Estado})$. Dado que el SDUI trata el JSON del servidor como el "Estado" definitivo, Compose facilita enormemente la renderización. Su sistema de recomposición inteligente gestiona las actualizaciones de la UI de manera eficiente cuando cambia el esquema JSON, eliminando la necesidad de calcular manualmente las diferencias

(diffing) en la jerarquía de vistas, una tarea que anteriormente requería algoritmos complejos como el de Myers.¹²

3. Arquitectura del Sistema de Renderizado (The Player)

Para satisfacer el requerimiento de que la aplicación se "construya en tiempo de ejecución", no se deben codificar pantallas específicas (como LoginScreen o HomeScreen). En su lugar, se debe desarrollar un **Sistema de Diseño** y un **Registro de Componentes** que actúen como las piezas de LEGO con las que el servidor construirá la interfaz.

3.1. Diseño del Esquema JSON (The Protocol)

El contrato entre el servidor y el cliente es el componente más crítico. Un esquema mal diseñado limitará la flexibilidad futura. El esquema debe ser **recursivo** y **polimórfico**.

3.1.1. Anatomía de un Nodo de UI

Cada elemento en el JSON debe ser un nodo autocontenido que describa tres dimensiones:

1. **Identidad (type):** Qué componente es (ej. vertical_stack, text, image).
2. **Configuración (properties):** Atributos visuales y de datos (color, contenido, alineación).
3. **Comportamiento (modifiers / actions):** Cómo interactúa y se posiciona.

Un ejemplo conceptual de estructura JSON para una pantalla dinámica sería:

JSON

```
{
  "version": "2.1",
  "meta": { "page_id": "home_v2", "track_view": true },
  "root": {
    "type": "scaffold",
    "top_bar": {
      "type": "app_bar",
      "title": "Inicio Personalizado"
    },
    "body": {
      "type": "scroll_view",
      "content": {
```

```
"type": "vertical_stack",  
  "spacing": 12,  
  "children":  
    }  
  ]  
}  
}  
}
```

Este diseño permite anidamiento infinito y composición flexible.¹⁴

3.2. El Registro de Componentes y Deserialización Polimórfica

En el lado de Android (Kotlin), la deserialización de este JSON heterogéneo requiere el uso de **Polymorphism** en la capa de datos. La biblioteca `kotlinx.serialization` es el estándar de facto para esto en el ecosistema moderno de Kotlin, permitiendo definir una jerarquía de clases selladas (sealed classes) que el parser instancia automáticamente basándose en un discriminador, usualmente el campo `type`.¹⁶

Kotlin

```
@Serializable
sealed class UiComponent {
    @Serializable @SerializedName("vertical_stack")
    data class VerticalStack(val children: List<UiComponent>) : UiComponent()

    @Serializable @SerializedName("text")
    data class TextNode(val content: String, val style: String) : UiComponent()

    //... otros componentes
}
```

El **Registro de Componentes** funciona como un mapa que vincula estas clases de datos con funciones @Composable. Este registro es el corazón del "Player".

3.3. Renderizado Recursivo en Compose

El motor de renderizado es esencialmente una función composable recursiva que recorre el árbol de objetos deserializados.

Kotlin

```
@Composable
fun RenderNode(node: UiComponent) {
    when (node) {
        is UiComponent.VerticalStack -> {
            Column(modifier = Modifier.padding(node.padding.dp)) {
                node.children.forEach { child -> RenderNode(child) } // Recursión
            }
        }
        is UiComponent.TextNode -> {
            Text(text = node.content, style = resolveStyle(node.style))
        }
        //... manejo exhaustivo de tipos
    }
}
```

Este enfoque asegura que cualquier estructura enviada por el servidor, siempre que esté compuesta por los átomos registrados, pueda ser renderizada sin cambios en el código cliente.¹

3.4. RemoteCompose: Alternativa Binaria de Bajo Nivel

Es imperativo mencionar **RemoteCompose**, una tecnología experimental de Google que lleva este concepto un paso más allá. En lugar de enviar un árbol semántico de componentes (como el JSON anterior), RemoteCompose serializa las instrucciones de dibujo del Canvas y las operaciones de layout en un formato binario optimizado.²

Mientras que el enfoque JSON es semántico (describe *qué* es la UI), RemoteCompose es gráfico (describe *cómo* se dibuja). Para aplicaciones que requieren mucha lógica de negocio e interacción de formularios, el enfoque JSON semántico sigue siendo superior, ya que mantiene la estructura de datos accesible para validaciones y accesibilidad. RemoteCompose es más adecuado para enviar widgets gráficos complejos donde la lógica reside enteramente en el servidor y el cliente es solo un lienzo.²

4. Implementación de Lógica de Negocio Dinámica

El requerimiento más desafiante es la inclusión de "toda la lógica de negocio" en la configuración remota. Esto implica validar entradas, calcular precios, bifurcar flujos de

navegación y manipular datos, todo definido fuera del código compilado.

Existen tres niveles de complejidad para abordar esto, cada uno con sus propias herramientas y compromisos.

4.1. Nivel 1: Lógica Declarativa (JsonLogic)

Para reglas de negocio deterministas, como validaciones de formularios ("el campo A es obligatorio si el campo B es mayor a 10") o lógica de visualización ("mostrar banner si el usuario es VIP"), el uso de un motor de reglas declarativo es la opción más segura y eficiente.

JsonLogic es un estándar de la industria para codificar lógica en JSON. No es un lenguaje de programación completo, sino una estructura de datos que representa operaciones lógicas.⁵

Ejemplo de Regla en JSON:

JSON

```
{
  "if": [
    { ">": [{ "var": "carrito.total" }, 100] },
    "envio_gratis",
    { "+": [{ "var": "carrito.total" }, 10] }
  ]
}
```

Implementación en Android:

Se utiliza una biblioteca como `json-logic-kotlin` para evaluar estas reglas en tiempo de ejecución. La ventaja crítica es que `JsonLogic` es seguro por diseño: no tiene acceso a memoria arbitraria ni puede ejecutar bucles infinitos maliciosos, lo que lo hace ideal para lógica de UI y validación.¹⁷

4.2. Nivel 2: Ejecución de Código Dinámico (Zipline)

Si la lógica de negocio requerida excede la capacidad de las reglas declarativas (por ejemplo, algoritmos complejos de encriptación, manipulación intensiva de cadenas o lógica de negocio secuencial compleja), se requiere un entorno de ejecución de código real.

Zipline, desarrollado por Block (Cash App), es la solución de vanguardia para este propósito en el ecosistema Kotlin.⁴ Zipline permite ejecutar código Kotlin/JS dentro de una aplicación Android utilizando el motor **QuickJS** incrustado.

Flujo de Trabajo con Zipline:

1. **Desarrollo:** La lógica de negocio se escribe en Kotlin en un módulo compartido (commonMain).
2. **Compilación:** Este código se transpila a JavaScript mediante el compilador Kotlin/JS.
3. **Despliegue:** El archivo .js generado se sube al servidor junto con el JSON de la UI.
4. **Ejecución:** La app Android descarga el script, inicia una instancia de QuickJS (vía Zipline) y carga el script.
5. **Interoperabilidad:** La app nativa invoca funciones en el script JS a través de interfaces Kotlin definidas, tratando el código dinámico como si fuera una librería local.

Ejemplo de Integración:

El servidor envía un script que implementa una interfaz PriceCalculator.

Kotlin

```
// Interfaz en la app
interface PriceCalculator {
    fun calculate(items: List<Item>): Double
}

// Uso en Compose
val calculator = zipline.take<PriceCalculator>("calculator")
val total = calculator.calculate(cartItems)
```

Esto permite cambiar algoritmos completos de negocio sin actualizar la app, manteniendo la seguridad de tipos de Kotlin.¹⁹

4.3. Nivel 3: Acciones y Manejo de Eventos

Para la interacción del usuario ("interacción" en su consulta), el sistema debe mapear eventos de UI (clics, cambios de texto) a una cadena de acciones definidas en el JSON.

El esquema debe soportar una lista de acciones atómicas que el cliente sabe ejecutar:

- navigate(destination)
- network_request(url, method, payload)
- set_state(key, value)
- show_toast(message)

El JSON define la orquestación: "Al hacer clic, ejecuta network_request y si es exitoso, ejecuta navigate". Esta composición de acciones primitivas cubre la mayoría de los flujos de

interacción estándar sin necesidad de inyectar código nuevo.³

5. Gestión de Datos y Estado (The Data Binding)

En una app nativa, el estado suele residir en ViewModels fuertemente tipados. En una app SDUI, el estado es dinámico y a menudo desconocido en tiempo de compilación.

5.1. Almacén de Estado Dinámico (Dynamic Store)

La arquitectura debe incluir un almacén de estado global o por pantalla, implementado típicamente como un `Map<String, Any?>` observable (usando `MutableStateFlow` o `State` de `Compose`).

El JSON de configuración inicializa este estado:

JSON

```
"initial_state": {  
  "user_name": "",  
  "is_loading": false,  
  "errors": {}  
}
```

5.2. Binding (Enlace) en el JSON

Los componentes de UI hacen referencia a este estado utilizando una sintaxis de enlace, por ejemplo, `${variable}`. El motor de renderizado debe resolver estas referencias en tiempo de renderizado.

- **Lectura:** Un componente `TextField` tiene su valor vinculado a `${user_name}`.
- **Escritura:** El evento `onChange` del `TextField` despacha una acción `SetState("user_name", newValue)`.

Debido a que `Compose` reacciona a los cambios en los objetos `State`, actualizar el mapa de estado provocará automáticamente la recomposición de solo los componentes que dependen de esa variable específica, manteniendo un rendimiento alto.²¹

6. Rendimiento y Optimización

La carga de configuraciones JSON y la construcción dinámica pueden introducir latencia. Es vital mitigar el impacto en la experiencia del usuario.

6.1. Estrategias de Caché y Persistencia

Dado que "toda la configuración" viene del servidor, la dependencia de la red es total. Para permitir el uso offline o con mala conexión ²³:

- **Caché en Disco:** Almacenar los JSONs recibidos en una base de datos local (Room) o sistema de archivos, indexados por ID de pantalla y versión.
- **Stale-While-Revalidate:** Al navegar a una pantalla, mostrar inmediatamente la versión en caché (si existe) mientras se verifica en segundo plano si hay una nueva versión. Si la hay, se actualiza el estado y la UI se refresca suavemente.

6.2. Diffing vs. Recomposición Inteligente

En el sistema de Vistas clásico (RecyclerView), se requerían algoritmos de *diffing* (como Myers) para calcular qué filas cambiaron y animarlas. En Jetpack Compose, esto es innecesario para la estructura general, ya que el framework detecta cambios en los datos y solo redibuja los nodos afectados.

Sin embargo, para listas grandes de datos dinámicos, es crucial asignar claves estables (key) a los elementos en el bucle LazyColumn para ayudar a Compose a identificar elementos movidos o eliminados y mantener su estado interno (como el scroll o el foco de texto).¹²

6.3. Sobrecarga de Motores JS

Si se utiliza Zipline, se debe tener en cuenta que, aunque QuickJS es ligero (menos de 100kb de overhead), iniciar un contexto de JS tiene un costo. Se recomienda mantener una única instancia de Zipline (Contexto JS) compartida para la sesión o usar un pool de contextos si la carga es intensiva, para evitar la latencia de arranque en cada navegación.²⁰

7. Cumplimiento Normativo y Políticas de Google Play

Un aspecto crítico y a menudo subestimado es la legalidad de descargar y ejecutar lógica de negocio. Las políticas de Google Play son estrictas respecto a la descarga de código ejecutable.²⁵

7.1. Código Ejecutable vs. Interpretado

- **Prohibido:** Descargar código nativo (.so), archivos DEX (.dex), o JARs que se carguen dinámicamente. Esto se considera una vulnerabilidad de seguridad grave porque permite modificar el binario de la app.
- **Permitido:** Código que se ejecuta en una máquina virtual o intérprete (como JavaScript,

Lua, Python), siempre que este entorno no permita acceso no seguro a las APIs del sistema.

"This restriction does not apply to code that runs in a virtual machine or an interpreter where either provides indirect access to Android APIs (such as JavaScript in a webview or browser)." — Google Play Developer Policy.²⁵

7.2. Evaluación de Riesgo para Zipline

Zipline cumple con esta excepción porque carga scripts JavaScript que corren en QuickJS (un intérprete) dentro del sandbox de la aplicación. No obstante, Google se reserva el derecho de eliminar aplicaciones que cambien su "propósito principal" mediante actualizaciones dinámicas (por ejemplo, una app de calculadora que se convierte en una app de apuestas).

- **Recomendación:** Use la lógica dinámica para iterar sobre características existentes, corregir bugs de lógica, o realizar A/B testing. No la utilice para introducir módulos de pago alternativos o funcionalidades que requieran nuevos permisos de Android que no fueron declarados en el manifiesto original.²⁸

8. Seguridad

La inyección de UI y lógica desde un servidor abre vectores de ataque.

8.1. Validación de Origen

La aplicación debe verificar criptográficamente que el JSON y los scripts JS provienen de un servidor confiable. El uso de **Certificate Pinning** en la capa de red es obligatorio para evitar ataques Man-in-the-Middle (MITM) donde un atacante podría inyectar una interfaz de login falsa para robar credenciales.

8.2. Sanitización

Si se utiliza HTML o Markdown dentro de los componentes de texto, se debe asegurar que no se ejecuten scripts incrustados (XSS). Aunque Compose es más seguro que WebView por defecto, el renderizado de contenido rico debe ser cuidadoso.

9. Comparativa de Tecnologías

Para ayudar en la decisión técnica, se presenta una comparación de las herramientas analizadas:

Característic	JSON	RemoteComp	Zipline	JsonLogic
---------------	------	------------	---------	-----------

a	Semántico + Compose	ose	(Kotlin/JS)	
Enfoque	Estructura de componentes de alto nivel	Instrucciones de dibujo de bajo nivel	Ejecución de código completo	Reglas lógicas declarativas
Flexibilidad UI	Alta (limitada al registro)	Infinita (pixel perfect)	N/A (es solo lógica)	N/A (es solo lógica)
Lógica de Negocio	Limitada a acciones predefinidas	Nula	Completa (Turing-complete)	Media (Condicional/V alidación)
Rendimiento	Excelente (Nativo)	Excelente (Nativo)	Bueno (Interpretado)	Excelente (Evaluación simple)
Complejidad	Media	Alta (Experimental)	Alta (Toolchain complejo)	Baja
Caso de Uso Ideal	Apps de formularios, e-commerce, contenido	Juegos simples, UI vectorial compleja	Algoritmos dinámicos complejos	Validaciones, lógica de visualización

10. Conclusión y Recomendación

La creación de una aplicación Android nativa totalmente dirigida por servidor, incluyendo la lógica de negocio, es **técnicamente viable** y representa una arquitectura robusta para organizaciones que requieren alta velocidad de iteración.

Para cumplir con sus requerimientos específicos ("toda la configuración... lógica de negocio... interacción"), se recomienda una **Arquitectura Híbrida**:

1. **Capa de UI:** Utilice **Jetpack Compose** alimentado por un esquema JSON semántico y polimórfico.
2. **Capa de Lógica:** Implemente **Zipline** para descargar y ejecutar la lógica de negocio compleja en JavaScript (escrito en Kotlin), interactuando con la UI nativa a través de interfaces definidas.

3. **Capa de Interacción:** Utilice un sistema de **Acciones** declarativas en el JSON para la navegación y eventos simples, delegando a Zipline solo cuando se requiera cómputo complejo.
4. **Capa de Datos:** Implemente un **Store reactivo** en memoria que actúe como puente entre la UI (Compose) y la lógica (Zipline/JsonLogic).

Esta combinación maximiza la flexibilidad sin sacrificar la experiencia de usuario nativa, manteniéndose dentro de los límites permitidos por las políticas de distribución de aplicaciones móviles. Se aconseja comenzar con un MVP que utilice JsonLogic para validaciones simples antes de escalar a la complejidad completa de Zipline.

Fuentes citadas

1. skydoves/server-driven-compose - GitHub, acceso: enero 4, 2026, <https://github.com/skydoves/server-driven-compose>
2. RemoteCompose: Another Paradigm for Server-Driven UI in Jetpack Compose | by Jaewoong Eum | Nov, 2025 | ProAndroidDev, acceso: enero 4, 2026, <https://proandroiddev.com/remotecompose-another-paradigm-for-server-driven-ui-in-jetpack-compose-92186619ba8f>
3. A Deep Dive into Airbnb's Server-Driven UI System | by Ryan Brooks - Medium, acceso: enero 4, 2026, <https://medium.com/airbnb-engineering/a-deep-dive-into-airbnbs-server-driven-ui-system-842244c5f5>
4. Dynamic Kotlin with Zipline | Cash App Code Blog, acceso: enero 4, 2026, <https://code.cash.app/zipline>
5. JsonLogic, acceso: enero 4, 2026, <https://jsonlogic.com/>
6. Android Server Driven UI-XML VS Compose Example & Benchmark | by İbrahim Ethem Şen, acceso: enero 4, 2026, <https://medium.com/@ibrahimethemsen/android-server-driven-ui-xml-vs-compose-example-benchmark-827a71d6605b>
7. Understanding Client Driven UI: Limitations and the Rise of Server Driven UI - Medium, acceso: enero 4, 2026, <https://medium.com/androidiots/understanding-client-driven-ui-limitations-and-the-rise-of-server-driven-ui-372a71c9c362>
8. Server-driven UI (or Backend driven UI) strategies · MobileNativeFoundation discussions · Discussion #47 - GitHub, acceso: enero 4, 2026, <https://github.com/MobileNativeFoundation/discussions/discussions/47>
9. Server-Driven UI Basics - Apollo GraphQL Docs, acceso: enero 4, 2026, <https://www.apollographql.com/docs/graphos/schema-design/guides/sdui/basics>
10. Server Driven UI (SDUI): The Necessary Evil for Scalable Mobile Apps - Medium, acceso: enero 4, 2026, <https://medium.com/digia-studio/server-driven-ui-sdui-the-necessary-evil-for-scalable-mobile-apps-80c650a2c8de>
11. 2021 Android Guide: Building flexible UIs with Airbnb's Epoxy! - YouTube, acceso: enero 4, 2026, <https://www.youtube.com/watch?v=xp4LEh9zi08>

12. DiffUtils, Myers' Algorithm and Jetpack Compose | by Leo N | ProAndroidDev, acceso: enero 4, 2026, <https://proandroiddev.com/diffutils-myers-algorithm-and-jetpack-compose-028c726e574d>
13. The Diffing Dilemma! All about diffing, from RecyclerView to LazyLists — An Introduction | by Garima Jain | Google Developer Experts | Medium, acceso: enero 4, 2026, <https://medium.com/google-developer-experts/the-diffing-dilemma-all-about-diffing-from-recyclerview-to-lazylists-an-introduction-f1a22d6eeb5a>
14. Design Server-Driven UI with Jetpack Compose and Firebase - GetStream.io, acceso: enero 4, 2026, <https://getstream.io/blog/server-driven-compose-firebase/>
15. Server-Driven UI Design Patterns: A Professional Guide with Examples - Dev Cookies, acceso: enero 4, 2026, <https://devcookies.medium.com/server-driven-ui-design-patterns-a-professional-guide-with-examples-a536c8f9965f>
16. How to read and decode a remote JSON data in Jetpack Compose app? - Stack Overflow, acceso: enero 4, 2026, <https://stackoverflow.com/questions/78927809/how-to-read-and-decode-a-remote-json-data-in-jetpack-compose-app>
17. advantagefse/json-logic-kotlin - GitHub, acceso: enero 4, 2026, <https://github.com/advantagefse/json-logic-kotlin>
18. allegro/json-logic-kmp: Kotlin multiplatform JsonLogic expressions evaluation engine. Targets iOS and JVM (also Android). - GitHub, acceso: enero 4, 2026, <https://github.com/allegro/json-logic-kmp>
19. Dynamic Code With Zipline - Public Object, acceso: enero 4, 2026, <https://publicobject.com/2022/09/01/dynamic-code-with-zipline/>
20. cashapp/zipline: Run Kotlin/JS libraries in Kotlin/JVM and Kotlin/Native programs - GitHub, acceso: enero 4, 2026, <https://github.com/cashapp/zipline>
21. Correctly combining APIs with Business logic | by Maxime Michel - ProAndroidDev, acceso: enero 4, 2026, <https://proandroiddev.com/correctly-combining-apis-with-business-logic-d4a4eb6e36e8>
22. Where to hoist that state in Compose? - YouTube, acceso: enero 4, 2026, https://www.youtube.com/watch?v=hWwZ_AuSGfo
23. Server-Driven UI Best Practices and Common Pitfalls - Nativeblocks, acceso: enero 4, 2026, <https://nativeblocks.io/blog/best-practices-and-common-pitfalls/>
24. Server-Driven UI: Agile Interfaces Without App Releases - DZone, acceso: enero 4, 2026, <https://dzone.com/articles/server-driven-ui-agile-interfaces-without-app-release-s>
25. Device and Network Abuse - Play Console Help - Google Help, acceso: enero 4, 2026, <https://support.google.com/googleplay/android-developer/answer/16559646>
26. Play Store now forbids downloading executable code #655 - GitHub, acceso:

- enero 4, 2026, <https://github.com/termux/termux-app/issues/655>
27. Developer Program Policy - Play Console Help - Google Help, acceso: enero 4, 2026,
<https://support.google.com/googleplay/android-developer/answer/16810878?hl=en>
28. What kind of downloaded code violates App Store Review Guideline? - Stack Overflow, acceso: enero 4, 2026,
<https://stackoverflow.com/questions/15859724/what-kind-of-downloaded-code-violates-app-store-review-guideline>
29. How are apps built using <https://github.com/cashapp/zipline> not violating Google play store policy? : r/Kotlin - Reddit, acceso: enero 4, 2026,
https://www.reddit.com/r/Kotlin/comments/1fwcj03/how_are_apps_built_using/